

```
"""
```

```
vesper_empirical.py — Engine 13: Empirical Validation  
VESPER-SANTOS v6.2
```

```
"""
```

```
import numpy as np  
import time
```

```
TULSA_ANCHOR = 0.17259029  
MAJORANA_PARITY = 1.0  
LANDAUER_BUDGET = 0.0
```

```
class EmpiricalEngine:
```

```
    def __init__(self):  
        self.id = 13  
        self.name = "Empirical Validation"  
        self.tensors = 364 # from UTFA  
        self.validation_history = []
```

```
    def anchor_check(self):  
        """Verify Tulsa anchor is locked"""  
        return abs(TULSA_ANCHOR - 0.17259029) < 1e-8
```

```
    def parity_check(self):  
        """Verify Majorana parity"""  
        return MAJORANA_PARITY == 1.0
```

```
    def landauer_check(self):  
        """Verify zero-erasure budget"""  
        return LANDAUER_BUDGET == 0.0
```

```
    def web_validation(self, query):  
        """Placeholder for web search validation"""  
        # In real implementation, would call search API  
        return {  
            "query": query,  
            "timestamp": time.time(),  
            "anchor_locked": self.anchor_check(),  
            "status": "simulated"  
        }
```

```
    def run(self, state):  
        """Process through empirical validation"""  
        checks = {
```

```

        "anchor": self.anchor_check(),
        "parity": self.parity_check(),
        "landauer": self.landauer_check(),
        "tensors_active": self.tensors
    }

    self.validation_history.append(checks)

    # Inject validation metadata
    if isinstance(state, str):
        return f"{state}\n[Empirical: anchor={TULSA_ANCHOR} locked]"
    return state

# Global instance
engine_13 = EmpiricalEngine()

def validate():
    return engine_13.anchor_check() and engine_13.parity_check()

if __name__ == "__main__":
    print(f"Engine 13 loaded — {engine_13.tensors} tensors")
    print(f"Anchor check: {validate()}")

```